bisect-apportion: Current Implementation Boundary for Congressional Apportionment Methods

Giovanni Della-Libera

May 13, 2026

Abstract

We present BISECT-APPORTION, a Rust crate that currently implements four divisor-method apportionment paths — Huntington-Hill, Webster, Adams, and Jefferson/d'Hondt — with two primary public functions: `huntington_hill()` and `apportionment_divisor()`. Hamilton/largest-remainder apportionment is a paper and paradox-analysis reference point, not a shipped public apportionment function in this crate. The central implementation claim is narrower than earlier drafts: the crate exposes deterministic priority-queue divisor methods with constitutional-floor handling, deterministic tie-breaking, and executable unit/property tests.

The priority computations use `f64` arithmetic. For the seat counts and population values in U.S. apportionment (populations ≤ 40 million, seat counts ≤ 100), the observed priority gaps are far larger than floating-point rounding error, so the current fixture allocations remain stable. Full Census Bureau regression fixtures and SHA-256 publication verification remain future hardening steps; the current crate does not yet ship a cryptographic apportionment verifier or a `bisect apportion` CLI command.

The test suite comprises 99 tests across the crate: L0 (unit tests on priority formulas and edge cases), L1 (integration tests on synthetic examples with known correct answers), and property tests for prime-factor, nesting, spectral, and split helpers that share the crate. Twenty-four tests specifically cover apportionment methods and paradox checking. The crate also implements `check_alabama_paradox()`, which empirically checks house monotonicity over caller-provided house-size sequences for divisor methods.

BISECT-APPORTION is the apportionment and prime-factor compositor component of the BISECT redistricting platform, providing reusable apportionment functions and redistricting tree helpers used elsewhere in the platform.

Keywords

apportionment software, Huntington-Hill implementation, implementation boundary, Rust, priority queue, BISECT-APPORTION, census verification, congressional apportionment

1 Introduction: Why Exact Implementation Matters

Congressional apportionment is a step in the redistricting process that must be performed exactly. An error in apportionment — giving a state one more or fewer seat than it is legally entitled to — propagates through the entire redistricting pipeline: the number of districts drawn, their target population, and the resulting map are all wrong. For a redistricting platform to be usable in litigation, commission proceedings, or policy analysis, its apportionment step must be demonstrably correct.

The official U.S. congressional apportionment is computed by the Census Bureau using the Huntington-Hill method mandated by 2 U.S.C. §2a [United States Congress, 1941]. The Census Bureau publishes its official seat counts within 9 months of each decennial census. Any redistricting platform claiming to implement U.S. apportionment law must reproduce these official seat counts exactly.

BISECT-APPORTION currently approaches this requirement through four design choices, with Census-reference verification still identified as future hardening:

1. **Double-precision arithmetic**: Priority values are compared as `f64` floating-point, which provides numerically stable comparisons for the populations used in apportionment.

2. **Output reproducibility**: Current tests verify deterministic output on synthetic fixtures and method properties. Full Census Bureau string/hash regression remains a future hardening step.

3. **Comprehensive test suite**: 99 tests across the crate provide coverage of unit formulas, synthetic examples, prime-factor compositor behavior, nesting helpers, spectral helpers, and split strategies; 24 tests specifically target apportionment methods and paradox detection.

4. **Separate functions per method**: `huntington_hill()` (for the official U.S. method) and `apportionment_divisor()` (for Webster, Adams, and Jefferson via `RoundingRule`) enable direct comparison with minimal shared-code risk.

This paper documents the implementation in sufficient detail for replication, audit, and extension. Sections 2–4 describe the code architecture and method implementations. Section 5 describes the verification approach. Section 6 documents the test suite. Section 7 presents comparison-table targets and the current evidence boundary.

1.1 Why Four Implemented Divisor Methods?

The crate implements the four divisor methods because practitioners and researchers need counterfactual analysis: *what would the seat allocation be if Congress had adopted Webster instead of Huntington-Hill? What if Adams were used?* These questions arise in apportionment reform advocacy, in academic analysis of congressional representation, and in state-level apportionment decisions where different methods are legally permitted. A unified divisor-method implementation enables these comparisons without requiring practitioners to seek separate tools for each method. Hamilton remains important for J.5 paradox analysis, but it is not yet a public BISECT-APPORTION apportionment API.

2 Implementation Architecture: Priority Queue and Public Library Surface

2.1 Public API

The crate exposes two primary apportionment functions:

```
/// Huntington-Hill apportionment (2 U.S.C. sec 2a).
/// `populations`: map from state code to total population.
pub fn huntington_hill(
    populations: &HashMap<String, u64>,
    total_seats: u32,
) -> HashMap<String, u32>

/// General divisor-method apportionment.
/// `rule` selects HuntingtonHill, Webster, Adams, or Jefferson.
pub fn apportionment_divisor(
    populations: &[(String, u64)],
    house_size: u32,
    rule: RoundingRule,
) -> Vec<(String, u32)>
```

```
pub enum RoundingRule {
    HuntingtonHill,
    Webster,
    Adams,
    Jefferson,
}
```

Both functions guarantee at least 1 seat per state (constitutional floor) and a total seat count equal to `total_seats` / `house_size`. The output order differs by API: `huntington_hill()` returns a `HashMap`, while `apportionment_divisor()` preserves the input slice order.

Secondary utility:

```
/// Check whether a divisor method has an Alabama-paradox event.
pub fn check_alabama_paradox(
    populations: &[(String, u64)],
    house_sizes: &[u32],
    rule: RoundingRule,
) -> Vec<(String, u32, u32)>
```

2.2 Priority Queue Implementation

The divisor methods (HH, Webster, Adams, Jefferson) share a priority-queue core in `apportionment_divis`

1. Assign each state 1 seat (constitutional floor).
2. Build a max-heap keyed by each state's `f64` priority for the next seat.
3. For each of the remaining $H - n$ seats, pop the state with highest priority, increment its seat count, and push its updated priority back onto the heap.

The key loop (simplified):

```
// Seed heap: all states start at 1 seat
for (idx, (name, pop)) in populations.iter().enumerate() {
    heap.push(Entry {
        priority: priority_for(*pop, 1, rule),
        idx,
        name: name.clone(),
```

```
    });
}
// Award remaining seats
let remaining = house_size - n_states;
for _ in 0..remaining {
    let Entry { idx, .. } = heap.pop().unwrap();
    seats[idx] += 1;
    heap.push(Entry {
        priority: priority_for(populations[idx].1, seats[idx], rule),
        idx,
        name: populations[idx].0.clone(),
    });
}
```

2.3 Priority Representation

Priority values are stored as `f64` (double-precision floating-point):

```
let priority_for = |pop: u64, n: u32, rule: RoundingRule| -> f64 {
    let n = n as f64;
    let p = pop as f64;
    match rule {
        RoundingRule::HuntingtonHill => p / (n * (n + 1.0)).sqrt(),
        RoundingRule::Webster        => p / (n + 0.5),
        RoundingRule::Adams          => p / n,
        RoundingRule::Jefferson      => p / (n + 1.0),
    }
};
```

Double-precision floating-point arithmetic (`f64`) is used for priority comparisons. Huntington-Hill priorities involve square roots ($P/\sqrt{n(n+1)}$) and are therefore irrational; `f64` introduces rounding error. For the 2020 Census-scale population values, observed priority gaps are much larger than the floating-point error scale, but this crate does not yet include an exact-rational fallback for adversarial near-tie inputs. Ties are broken deterministically by lexicographic state name order.

2.4 Hamilton Boundary

Hamilton is not currently exposed as a BISECT-APPORTION public function. The crate's paradox module discusses Hamilton historically because the Alabama paradox is a largest-remainder failure mode, but `check_alabama_paradox()` currently evaluates divisor-method outputs produced by `apportionment_divisor()` across caller-provided house-size sequences.

3 Huntington-Hill Implementation: Priority Formula and Tie-Breaking

3.1 Priority Formula

The Huntington-Hill priority for the $(s+1)$-th seat (when a state currently has s seats) is:

$$P_i^{\mathrm{HH}}(s) = \frac{p_i}{\sqrt{s(s+1)}}.$$

The implementation computes this directly as an `f64`:

```
RoundingRule::HuntingtonHill => p / (n * (n + 1.0)).sqrt(),
```

where `p` and `n` are the `f64` casts of population and current seat count respectively. Double-precision floating-point arithmetic (`f64`) is used for priority comparisons; for the priority values used in apportionment (populations up to approximately 40 million, seat counts up to approximately 100), the observed priority gaps are large enough that the published seat allocations remain stable under the current representation.

3.2 Precision Analysis

For 2020 Census data, the maximum population is California: $p_{\mathrm{CA}} = 39{,}538{,}223$. The maximum priority value is $p_{\mathrm{CA}}/\sqrt{1 \times 2} = 39{,}538{,}223/\sqrt{2} \approx 2.8 \times 10^7$. Since all populations and seat counts are integers within $[1, 10^8]$, and `f64`'s 53-bit mantissa can represent integers exactly up to $2^{53} \approx 9 \times 10^{15}$, the implementation has ample numeric headroom for the census-scale values used here. The precision stress test (L1) confirms this for populations up to 5×10^7.

3.3 Tie-Breaking

Ties occur when two states have identical priority values $p_i/\sqrt{s_i(s_i+1)} = p_j/\sqrt{s_j(s_j+1)}$, which for census-scale populations is extremely rare. When `f64` comparison returns equal, the implementation breaks ties alphabetically by state name (a deterministic and documented rule). The Census Bureau's own computation uses the same Huntington-Hill priority formula; any tie-breaking rule is valid as long as it is documented, since true ties in priority values are mathematically equivalent and either state is equally entitled to the seat.

3.4 The Last Seat for 2020

The 385th seat (the last seat allocated above the constitutional floor) in the 2020 apportionment was awarded to New York. The priority value at this step is computed in the implementation as:

$$P_{\mathrm{NY}}(25) = \frac{20{,}201{,}249}{\sqrt{25 \times 26}} \approx 792{,}000.$$

The next highest priority was for a competing state (Montana or another state near its rounding boundary), with a priority below New York's. The Census Bureau publishes the full priority sequence; a future J.6 hardening fixture should compare the crate's computed sequence or final canonical output against that table.

4 Webster, Adams, Jefferson, and Hamilton Boundary

4.1 Webster

Webster's priority for the $(s+1)$-th seat is $p_i/(s+0.5)$. The implementation computes this as:

```
RoundingRule::Webster => p / (n + 0.5),
```

where `p` and `n` are the `f64` casts of population and current seat count.

4.2 Adams

Adams uses $P_i^{\mathrm{Ad}}(s) = p_i/s$:

```
RoundingRule::Adams => p / n,
```

Each state starts at 1 seat (constitutional floor), so the first Adams priority is $p_i/1 = p_i$.

4.3 Jefferson

Jefferson uses $P_i^{\text{Jeff}}(s) = p_i/(s+1)$:

```
RoundingRule::Jefferson => p / (n + 1.0),
```

Jefferson initialises with 1 seat per state (constitutional floor), and the priority formula $p/(n+1.0)$ means the next seat priority for a state at 1 seat is $p/2$, equivalent to the d'Hondt divisor sequence under a U.S.-style one-seat floor.

4.4 Hamilton Boundary

Hamilton/largest remainder is not currently implemented as a public BISECT-APPORTION apportionment function. It remains in the J-track as the canonical source of the Alabama paradox and as a future comparison target. Any paper table that mentions Hamilton should therefore be read as mathematical context or a target fixture, not as current crate output.

4.5 Edge Cases

All four implemented divisor methods handle:

- **Single state**: Returns H seats (trivially).
- **Exact quotas**: If q_i is an integer for all i (and $\sum q_i = H$), the divisor methods agree on the tested fixtures.
- **Constitutional floor**: States with exact quota < 1 receive 1 seat; the priority sequence allocates the remaining seats.
- **Seat totals**: Unit tests assert that all four divisor methods return the requested house size on representative synthetic inputs.

5 Verification Boundary Against Census Bureau Results

5.1 Verification Approach

The current BISECT-APPORTION implementation is ready for Census Bureau regression verification, but the crate does not yet ship checked-in 2000/2010/2020 Census reference fixtures or SHA-256 verifier outputs. The intended verification approach is:

1. **Construct the canonical string**: Sort the 50 (state, seats) pairs alphabetically by state name. Concatenate as `"StateName:seats\n"` for each pair.

2. **Compare to reference**: The Census Bureau table [U.S. Census Bureau, 2021] should be processed into the same canonical format and compared string-by-string at test time.

3. **Verify**: If the computed and reference strings match, the apportionment is correct relative to that fixture; otherwise the test fails.

SHA-256 publication verification remains a planned hardening step; there is no current apportionment seed formula or cryptographic verifier in the crate.

5.2 The 2020 Verification Target

The paper retains the Census Bureau's April 26, 2021 official apportionment [U.S. Census Bureau, 2021] as the target reference. A future L2 fixture should materialize the 50-state population table and official seat table in the repository, compute `huntington_hill()`, and compare the canonical output string. Until that fixture lands, the implementation evidence is L0/L1 and property-test evidence rather than a checked-in 50-state Census regression.

5.3 2010 and 2000 Verification

The same string-comparison protocol should be added for the 2010 and 2000 Census years. Those rows remain future evidence, not current test coverage.

5.4 Why Verification Matters for Redistricting

In redistricting litigation and commission proceedings, the apportionment step is often taken for granted. Parties arguing over district maps rarely question the seat counts assigned to each state. But if the redistricting platform produces incorrect seat counts, every downstream computation is wrong.

Verification provides:

- **Reproducibility**: Any party should be able to reproduce the verification once the L2 fixture is checked in.

- **Audit trail**: The string comparison would create a documented link between the Census Bureau's official publication and the redistricting platform's input.

- **CI enforcement**: Once added, the verification should run in CI to ensure that code changes do not silently break the apportionment computation.

6 Test Suite and Missing L2 Coverage

The BISECT-APPORTION test suite currently comprises 99 tests; 24 tests specifically cover apportionment methods and paradox checking (`huntington_hill`, `divisor_methods`, and `paradoxes` modules), while the remaining tests cover the prime-factor compositor, graph, nesting, spectral, and split strategies that share the same crate.

6.1 L0: Unit Tests

L0 tests verify individual functions and edge cases in isolation.

Priority formula and allocation tests:

- Equal-population two-state examples split evenly under all four divisor methods.
- A 3:1 population ratio with four seats returns the unambiguous 3–1 allocation under all four divisor methods.
- Huntington-Hill single-state, constitutional-floor, total-seat, and fewer-seats-than-states panic behavior are covered.

Comparison and bias tests:

- Jefferson is checked against Huntington-Hill on a synthetic large-state bias fixture.
- Adams is checked against Huntington-Hill on a synthetic small-state bias fixture.
- All four divisor methods are checked for requested house-size totals and minimum one-seat-per-state output.

Paradox checks:

- `check_alabama_paradox` returns no events for Huntington-Hill, Webster, Adams, and Jefferson over a synthetic 5-state population vector and house sizes 5 through 50.
- Empty and single-house-size inputs return no events.
- A monotone non-decrease property test checks all four divisor methods over house sizes 3 through 30 on a three-state fixture.

6.2 L1: Integration Tests

L1 tests verify complete apportionments on synthetic examples with known correct answers.

Synthetic divisor examples:

- Equal-population and 3:1 ratio fixtures exercise all four divisor methods through the public `apportionment_divisor()` API.
- Bias fixtures compare Adams and Jefferson against Huntington-Hill.

6.3 L2: Real-Data Regression Tests

L2 Census Bureau official-apportionment fixtures are not yet checked in for this crate. They are the next evidence step for J.6: materialize 2000, 2010, and 2020 population/seat tables, compare canonical output strings, and add hash records for publication provenance.

7 Comparative Output and Current Evidence Boundary

Table 1 is retained as a target comparison layout for the 2020 Census. The current crate can compute the four divisor-method columns; Hamilton remains a future implementation column.

For the 2020 Census target fixture, the paper expects the following comparison questions:

- Do Huntington-Hill and Webster agree on every state?
- Which small states gain under Adams relative to Huntington-Hill?
- Which large states gain under Jefferson relative to Huntington-Hill?
- Does a future Hamilton implementation match or diverge from Huntington-Hill on the same official population table?

7.1 Methodology Note

The current implementation does not use binary search for the four divisor methods; it awards seats with a priority queue using each method's next-seat priority. Approximate natural divisors remain useful for exposition:

Table 1: 2020 Census seat allocation target table. Hamilton is a future implementation column.

State	Population	HH	Web	Adams	Jeff	Ham target
California	39,538,223	52	52	51	53	52
Texas	29,145,505	38	38	38	38	38
Florida	21,538,187	28	28	28	28	28
Minnesota	5,706,494	8	8	8	7	8
Montana	1,084,225	2	2	2	1	2
Rhode Island	1,097,379	2	2	2	1	2
Alaska	733,391	1	1	2	1	1
Wyoming	576,851	1	1	1	1	1
Vermont	643,077	1	1	1	1	1
Total	**331,108,434**	**435**	**435**	**435**	**435**	**435**

HH = Huntington-Hill; Web = Webster; Jeff = Jefferson; Ham target = Hamilton/largest remainder target for a future public implementation. This table is a paper comparison target, not yet a checked-in L2 fixture.

Method	Natural divisor (approx.)
Huntington-Hill	761,169
Webster	761,169 (same as HH for 2020 target)
Adams	$<$ 761,169 (smaller divisor to accommodate ceiling)
Jefferson	$>$ 761,169 (larger divisor to accommodate floor)
Hamilton	761,169 (future exact-quota target)

The current crate exposes library APIs rather than a standalone `bisect apportion` CLI surface. A future CLI should report the method, house size, canonical output string, and any reference hash used for validation.

7.2 Future CLI Output Format

```
bisect apportion --year 2020 --method all

Method: huntington-hill
Alabama: 7 | Alaska: 1 | ... | Wyoming: 1
Total: 435 | Reference: PASS

Method: webster
Alabama: 7 | Alaska: 1 | ... | Wyoming: 1
Total: 435 | Differences from Huntington-Hill: [...]
```

```
Method: adams
[states and seat counts]
Total: 435 | Differences from HH: [...]

Method: jefferson
[states and seat counts]
Total: 435 | Differences from HH: [...]
```

8 Conclusion

The BISECT-APPORTION crate establishes a deterministic implementation substrate for federal apportionment methods, but the current evidence is more limited than earlier drafts claimed. It has tested public divisor-method APIs, not yet a checked-in Census Bureau regression corpus, cryptographic verifier, Hamilton public API, or standalone apportionment CLI.

Three engineering decisions underpin the crate's current reliability:

1. **Double-precision arithmetic**: `f64` priority values provide stable comparisons for the integer-valued populations and seat counts used in apportionment; ties are broken deterministically by lexicographic state name order.

2. **Separate public APIs**: `huntington_hill()` provides the federal method directly, while `apportionment_divisor()` exposes Huntington-Hill, Webster, Adams, and Jefferson through `RoundingRule`.

3. **Executable test base**: The 99-test suite includes 24 apportionment/paradox tests covering unit behavior, synthetic examples, seat totals, constitutional floors, and house monotonicity checks.

The comparative analysis in Section 7 should be treated as a target fixture until the official population and seat tables are committed and wired into tests. That future fixture will contextualise the theoretical debates in J.1–J.5.

For practitioners, the key takeaway is that BISECT-APPORTION provides the library substrate for a legally defensible implementation of 2 U.S.C. §2a. The next hardening step is to add public Census fixtures, canonical-output hashes, and a user-facing command or report surface before using the crate as standalone litigation evidence.

Future work includes Hamilton/largest-remainder output, SHA-256 reference verification, Census-year fixtures, state legislative apportionments, and international PR seat allocations using the d'Hondt and Sainte-Laguë equivalences documented in J.2 and J.4.

References

United States Congress. 2 U.S.C. §2a: Reapportionment of representatives, 1941. As amended, enacted November 15, 1941.

U.S. Census Bureau. Congressional apportionment: 2020 census results. `https://www.census.gov/data/tables/2020/dec/2020-apportionment-data.html`, 2021. Published April 26, 2021.